

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

Debugging or Optimization task

Code of Conduct:

I Prathika/192421357 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Prathika

Signature of the student:

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)

Process P2:
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)

Identify the exact condition that causes deadlock.
Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.
You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread

- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Q6. Producer–Consumer Debugging (Semaphore Error)

A producer-consumer application occasionally hangs after running for several minutes.

Semaphore empty = 5

Semaphore full = 0

Semaphore mutex = 1

Producer:

```
wait(mutex)
wait(empty)
insert(item)
signal(full)
signal(mutex)
```

Consumer:

```
wait(mutex)
wait(full)
remove(item)
signal(empty)
signal(mutex)
```

The program freezes even though both producer and consumer threads are alive.

1. Identify the synchronization error causing the hang.
2. Explain why the semaphore ordering is incorrect.
3. Suggest the correct semaphore sequence to eliminate the problem.

(10 Marks)

Q7. Thread Performance Debugging

A web server creates one new thread for every incoming client request.

Performance statistics:

- Average requests/sec = 15,000
- Average thread creation time = 0.4 ms
- CPU utilisation = 92%
- Memory usage continuously increases

Developers notice that the server becomes slow after several hours.

1. Identify the thread-management problem.
2. Explain why the current multithreading model causes performance degradation.
3. Propose two optimisations to improve scalability.

(10 Marks)

Q8. Banker's Algorithm Debugging

Available Resources:

A B C = 2 1 1

Allocation:

P1: 1 0 1

P2: 0 1 0

P3: 1 1 1

Maximum:

P1: 2 1 2

P2: 1 2 1

P3: 3 2 2

Process P2 requests (1,1,1).

1. Calculate the Need matrix.
2. Determine whether the request can be granted immediately using Banker's Algorithm.
3. If not, identify the unsafe condition and explain why granting the request could lead to deadlock.

(10 Marks)

Q9. Deadlock Detection and Recovery Debugging

A database server reports that four transactions are waiting for resources.

Resource Allocation Graph:

- T1 waits for R2
- T2 waits for R3
- T3 waits for R4
- T4 waits for R1
- R1 allocated to T1
- R2 allocated to T2
- R3 allocated to T3
- R4 allocated to T4

The database becomes completely unresponsive.

1. Identify whether a deadlock exists.
2. Explain how deadlock detection identifies this situation.

3. Suggest two recovery techniques and justify which one minimises system disruption.

(10 Marks)

Q10. Monitor-Based Synchronization Debugging

A monitor is used to synchronize access to a shared printer.

```
monitor Printer
{
    boolean busy = false;

    procedure Print()
    {
        if(busy)
            wait();

        busy = true;

        // Printing

        busy = false;
    }
}
```

Users report that some print jobs never execute even though the printer becomes free.

1. Identify the logical error in the monitor implementation.
2. Explain why waiting processes are never awakened.
3. Modify the monitor logic to ensure correct synchronization and fairness.

(10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging(25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (100)
Q1						
Q2						
Q3						
Q4						
Q5						
Q6						
Q7						
Q8						

Q9						
Q10						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. DEADLOCK DEBUGGING:

GIVEN CODE :

P1(Error)

P2(Debugged)

lock(R1)

lock(R2)

wait(100ms)

wait(100ms)

lock(R2)

lock(R1)

i) Exact Condition using Deadlock:

Time

P1

P2

t1

LocksR1

t2

LocksR2

t3

waitsforR2

t4

waitsforR1

Explanation of Debug:-

- * Both process now acquire resources in the same order R1 → R2
- * There is no circular wait.
- * Deadlock is eliminated.

Q2. CPU UTILIZATION:

ERROR CODE:

Ready
↓
Running
↓
Waiting (I/O)
↓
Ready

DEBUGGED CODE:

```
* while(process_has_work)
{
    submitAsyncIO(); // Non blocking I/O
    executeCPUwork(); // CPU continues
    wait for IO completion();
}
```

Debug Explanation:

* Instead of blocking immediately

* The process performs useful computation while the I/O executes parallel.

* Hence the CPU utilization increases through this.

3. MEMORY BOTTLENECK:

ORIGINAL ERROR CODE:

```
createThread();
createThread();
createThread();
...
25 Threads.
```

DEBUGGED CODE:

```
ThreadPool pool(8);
while(tasksAvailable)
{
    pool.execute(task);
}
```

bug Explanation :-

Instead of creating many threads, 8 worker threads are reused.

Advantages:

* Lower memory usage.

* Fewer page faults.

* Better cache locality.

4. STARVATION (Scheduler)

ERROR CODE :

Q1 always executes

Q2 waits

Q3 never runs

DEBUGGED CODE :

```
for (each process)
{
    if (waitTime > LIMIT)
    {
        increasePriority(process);
    }
}
```

debug Explanation:

Long waiting processes gradually receive higher Priority.

Eventually P5 reaches Q2 or Q1 and gets CPU time.

95. CONTEXT SWITCHING:

ERROR CODE:

Quantum = 1ms

Burst = 2ms

DEBUGGED CODE:

```
Timequantum = 5ms;  
schedule (process);  
while (IO pending)  
{  
    useAsyncIO();  
}
```

DEBUG EXPLANATION:

- * larger time quantum.
- * less preemption.
- * Fewer context switches.

5. PRODUCER - CONSUMER (semaphore)

ERROR CODE:

```
wait(mutex);  
wait(empty);  
insert(item);  
signal(full);  
signal(mutex);
```

DEBUGGED CODE:

```
wait(empty); // wait for empty slot 1st  
wait(mutex); // Then lock buffer  
insert(item);  
signal(mutex); // Release mutex 1st  
signal(full); // Notify consumer.  
  
wait(full); // wait until item exists.  
wait(mutex); // lock buffer  
remove(item);  
signal(mutex); // unlock  
signal(empty); // notify producer
```

is threads only locks the critical section after confirming the required resource (empty/full slot).
This prevents a blocked thread.

1. THREAD PERFORMANCE:-

ORIGINAL :

```
onclientrequest()  
{  
  create Thread();  
  handle client();  
  destroy Thread();  
}
```

DEBUGGED:

```
ThreadPool pool(50);  
onclientserver()  
{  
  pool.submit(handle client);  
}
```

DEBUG EXPLANATION:

- * Worker threads are reused.
- * No repeated → Thread creation.
→ Thread destruction.

Q8. BANKERS ALGORITHM:

"The mentioned code doesn't require debugging".

Debug Explanation :

- * Always perform the safety test before permanently allocating resources.

99. DEADLOCK DETECTION:

DEBUGGED CODE:

```
if (deadlock detected())  
{  
    victim = select lowest priority();  
    terminate(victim);  
}
```

Debug Explanation:

- * Breaking the cycle by terminating or rolling back one transaction releases resources and allows the remaining transactions to complete.

10. MONITOR SYNCHRONIZATION:

Debugged code:

```
* monitor printer  
{  
    boolean busy = false;  
    condition printer available;  
    procedure print()  
    {  
        while (busy)  
        {  
            cwaitforprinteravailable();  
        }  
        busy = true;  
        // printing  
        busy = false;  
        signal(cforprinteravailable);  
    }  
}
```

Debug Explanation:

- * while is used instead of if to recheck the condition after waking.
- * Signal(Printer available) wakes a waiting process when the printer becomes free.
- * Waiting printing jobs are no longer blocked indefinitely.